

Unit 4 Case Studies

Distributed Models: P2P

Case Study: Amazon Aurora – Distributed Database with P2P-Like Replication Architecture

Context: Amazon Web Services (AWS), 2014–present. Aurora is AWS's proprietary relational database engine compatible with MySQL and PostgreSQL, designed for cloud-native applications requiring high availability and performance at scale. It serves thousands of enterprises and SaaS providers managing petabytes of data globally.

Problem: Traditional database replication architectures faced fundamental limitations:

- **Read-write coordination overhead:** Single-node primary bottlenecks on write operations
- **Failover latency:** Recovering from primary instance failure required time-consuming recovery and failover processes, impacting availability
- **Storage efficiency:** Full data replication to each replica instance consumed excessive storage and caused I/O amplification
- **Availability zones isolation:** Traditional replication couldn't efficiently leverage multiple availability zones for both redundancy and performance

The challenge was designing a distributed database that could achieve high throughput, low-latency failover, and durability without the overhead of traditional master-replica architectures.

Solution: Aurora implements a **P2P-style distributed architecture** with the following key innovations:

1. **Shared Distributed Storage Layer:** Aurora separates compute from storage, using a shared cluster volume that replicates data across three Availability Zones automatically. Multiple database instances (primary + up to 15 Aurora Replicas) act as peers connecting to the same logical storage volume—eliminating per-replica storage replication.
2. **Quorum-Based Write Consistency:** Writes use a quorum-based approach where the primary writes to the distributed storage layer, and consensus is achieved across AZs. This P2P-like coordination ensures durability without synchronous replication delays.
3. **Read-Only Replicas as Peers:** Aurora Replicas are fully independent read-only instances that connect to the same cluster volume. They act as distributed peers in the architecture:
 - Each replica sees the same data with minimal lag (typically < 100ms)
 - Any replica can be promoted to primary (fast failover in seconds, not minutes)

- Read workloads are distributed across replicas using the reader endpoint, distributing load peer-to-peer
4. **Minimal Data Replication:** Unlike traditional replication where each replica gets a full copy, Aurora's shared storage means:
- One data copy spans AZs (eliminating I/O amplification)
 - Replicas are added instantly without copying data
 - Storage scales independently from the number of instances

Results/Lessons:

- **5x throughput increase:** Aurora achieves 5x the throughput of traditional MySQL and 3x of PostgreSQL by optimizing network round trips through its P2P storage coordination
- **Sub-second failover:** Automatic failover to Aurora Replicas completes in seconds (vs. minutes for traditional approaches), enabling near-continuous availability (99.99% SLA for multi-AZ, 99.999% with global database)
- **Cost efficiency:** By eliminating per-replica storage duplication, Aurora reduces storage costs by up to 90% compared to traditional setups while improving performance
- **Scalability:** Deployments can handle petabyte-scale data with automatic storage scaling, supporting thousands of customers running mission-critical workloads
- **Industry adoption:** Companies like Samsung (migrated 1.1 billion users), BMW, and DraftKings rely on Aurora's P2P-distributed architecture for critical workloads

Key Insight: Aurora demonstrates that P2P-inspired distributed architectures—where multiple compute peers coordinate through a shared, consensus-based storage layer—outperform traditional hierarchical replication in cloud environments. By treating replicas as true peers rather than subordinates, Aurora achieved better scalability, resilience, and cost-efficiency than previous database designs.

References:

- <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Overview.html>
- <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Replication.html>
- <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Overview.StorageReliability.html>

Distributed Models: Client-Server

Case Study: Hadoop Distributed File System (HDFS) – Large-Scale Data Processing Infrastructure

Context: Apache Hadoop (2006–present), an open-source framework used by enterprises worldwide for big data processing. Originally developed at Yahoo to support web indexing,

Hadoop powers data infrastructure at LinkedIn, Facebook, Netflix, Twitter, and thousands of organizations processing petabytes of data daily. Hadoop is the foundation of enterprises' batch processing and data warehouse platforms.

Problem: Traditional centralized file systems cannot handle the scale and fault-tolerance requirements of big data:

- **Single point of failure:** Centralized servers become bottlenecks and critical failure points
- **Lack of scalability:** Traditional storage cannot grow to petabyte scale cost-effectively
- **Data locality:** Moving terabytes across networks is prohibitively slow; computation must move to data
- **Fault tolerance:** Hardware failure is guaranteed in clusters of thousands of machines; recovery must be automatic
- **Write-once semantics:** Most big data applications don't require fine-grained updates; a simpler consistency model can improve performance

Hadoop needed to design a distributed system where thousands of commodity machines could collaborate without a powerful central authority.

Solution: Hadoop implements a **strict client-server master/slave architecture** via its HDFS component:

1. NameNode (Master):

- Single authoritative server managing the entire file system namespace
- Maintains the file system tree and metadata for all files/directories
- Does NOT store actual data (avoiding bottleneck)
- Clients send ALL metadata requests (open, close, rename files) to the NameNode via **ClientProtocol**
- NameNode decides which DataNodes store data blocks and maintains replica mappings
- Acts as the **central orchestrator** for the entire cluster

2. DataNodes (Slave/Worker Servers):

- Hundreds to thousands of worker nodes, each storing data blocks (default 128-256MB per block)
- Perform read/write operations on behalf of clients (no involvement of NameNode in data operations)
- Send **Heartbeat messages** to NameNode every 3 seconds (proving they're alive)
- Send **BlockReports** periodically listing all blocks they store
- Follow NameNode instructions for block replication, deletion, and rebalancing

3. Client-Server Communication Protocol:

- Clients communicate with NameNode via **RPC (Remote Procedure Call) over TCP/IP**
- All data operations bypass the NameNode: clients get block locations from NameNode, then read/write directly from DataNodes
- This separation ensures the NameNode doesn't become a data bottleneck

4. Robust Fault Tolerance:

- **Heartbeat timeout:** If a DataNode misses heartbeats for >10 minutes, NameNode marks it dead
- **Automatic re-replication:** NameNode automatically replicates blocks from failed nodes to others
- **Rack awareness:** Replicas are distributed across racks to survive rack failures
- **Replication pipeline:** Data flows from client → DataNode1 → DataNode2 → DataNode3 simultaneously

Results/Lessons:

- **Scalability to petabyte-scale:** Hadoop clusters grew from hundreds to thousands of nodes, handling exabytes of data
- **Cost-effectiveness:** Uses commodity (cheap) hardware; no need for specialized enterprise storage
- **Fault-tolerance in practice:** LinkedIn, Facebook, and Twitter deployed Hadoop clusters with thousands of nodes, experiencing regular hardware failures with zero data loss
- **Data locality optimization:** By moving computation to data (MapReduce jobs scheduled on DataNodes), Hadoop achieved 10-100x network bandwidth savings
- **Simplified consistency model:** Write-once-read-many semantics allowed Hadoop to handle concurrent operations safely without complex locking
- **High throughput:** Batch jobs achieve gigabytes-per-second aggregate throughput despite individual node failures

Key Insight: Hadoop demonstrates that a **clear master/slave client-server hierarchy** is highly effective for distributed systems where:

- One entity (NameNode) can reliably manage metadata and orchestration for thousands of nodes
- Data operations are decoupled from metadata (clients talk directly to DataNodes)
- Fault tolerance is built through redundancy and automatic recovery, not consensus protocols
- The system prioritizes **throughput over latency** and **write-once over general transactions**

Unlike peer-to-peer systems that avoid central coordination, Hadoop's centralized NameNode proved that a single, well-designed master can coordinate massive scale while remaining highly available through careful design and replication of metadata.

References:

- <https://hadoop.apache.org/docs/r3.3.1/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/ClusterSetup.html>
- <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html>

Cloud Resource Management and Scheduling

Case Study: LY Corporation – Consolidating 164 OpenStack/Nova Clusters into Unified Platform

Context: LY Corporation (formed in 2023 from merger of Yahoo! Japan and South Korean messaging giant LINE) operates cloud infrastructure serving 300 million monthly users across e-commerce, payments, messaging, and portal services in Asia. The company maintains one of the world's largest private cloud deployments with millions of VMs and tens of thousands of physical servers.

Problem: LY Corp faced a critical resource management and scheduling challenge stemming from infrastructure fragmentation:

- **164+ OpenStack clusters:** Yahoo! Japan's "YNW" cloud ran more than 160 OpenStack clusters managing over 27,000 servers
- **Excessive customization:** Heavy custom modifications to OpenStack made cluster upgrades extremely difficult, keeping systems stuck on outdated versions
- **VM allocation inefficiency:** Sprawling clusters (LINE's "Verda" cloud alone had 130,000 VMs across 11,000 hosts and 4 clusters) suffered from poor resource utilization and scheduling inconsistency
- **Operational overhead:** Multiple cluster management strategies complicated resource scheduling decisions and capacity planning
- **Security and stability risks:** Old versions exposed infrastructure to security vulnerabilities; custom patches prevented timely security updates

The core challenge was optimizing Nova's scheduler decisions across fragmented infrastructure while maintaining upstream compatibility for continuous security and feature updates.

Solution: LY Corp designed "Flava," a next-generation unified cloud platform that exemplifies advanced resource management through:

1. **Single Unified Cluster Architecture:** Consolidating 164+ separate clusters into **one OpenStack cluster** running 500+ hosts and 9,000+ VMs. This unified architecture allows Nova's Filter Scheduler to make globally optimal VM allocation decisions instead of per-cluster sub-optimal placements.

2. Upstream-First Scheduler Configuration:

- Adopting vanilla OpenStack Nova scheduler with minimal customization
- Enabling pre-filters and proper resource placement queries to optimize VM allocation
- Using Placement service (Nova's resource inventory tracking) for accurate resource availability reporting
- Implementing ComputeFilter and ComputeCapabilitiesFilter for intelligent host selection

3. Stateless VM Design with Rapid Scheduling:

- Treating VM root disks as temporary/ephemeral, moving persistent data to external Ceph storage
- Enabling faster VM scheduling and reallocation without state-migration overhead
- Allowing Nova scheduler to treat all hosts as equivalent and schedule aggressively

4. Infrastructure-as-Code Driven Scheduling:

- Building recovery workflows around rapid VM redeployment rather than preserving exact state
- Allowing Nova scheduler flexibility to place VMs on any available resources without geographic/state constraints
- Reducing scheduler complexity by removing state-preservation requirements

5. Hardware Failure Automation:

- Automated failure detection and reintegration into Nova's resource pool
- Continuous hardware inventory updates feeding into Nova's Placement API
- Prometheus and Grafana monitoring for early anomaly detection before Nova receives bad node reports

6. Observability and Scheduler Tuning:

- Kernel-level tracing and packet capture to verify Nova scheduler decisions are correct
- Continuous monitoring of cloud health to validate allocation strategies
- Data-driven tuning of filter_scheduler parameters based on workload patterns

Results/Lessons:

- **Massive consolidation:** Reduced 164+ clusters to 1, enabling centralized Nova scheduler optimization
- **10x+ hosts per cluster reduction:** Achieved 500+ hosts vs. 27,000 servers previously sprawled across 160 clusters—reducing per-node scheduling complexity
- **Continuous security:** Upstream-aligned OpenStack enabled regular patching and feature updates (previously blocked by custom patches)
- **Improved resource utilization:** Unified cluster scheduling improved VM-to-host density and eliminated over-provisioning across fragmented clusters
- **Operational simplicity:** Single Nova scheduler to manage and tune instead of 164+ independent cluster schedulers

- **Cost efficiency:** Consolidated infrastructure, fewer management overhead, optimized VM placement
- **Scale mastery:** Successfully managing 300 million user services on unified infrastructure with unified scheduling

Key Insight: LY Corp demonstrates that **consolidating multiple resource management domains into a single unified scheduler** (Nova's Filter Scheduler) and **aligning closely with upstream implementations** yields superior resource optimization. By eliminating custom patches that blocked updates and consolidating fragmented clusters, Nova's scheduler algorithms could operate on a global view of resources, making decisions that individual per-cluster schedulers could never achieve. Upstream compatibility also enabled continuous improvement through OpenStack release cycles.

Reference: https://www.theregister.com/2026/04/07/ly_corp_openstack_consolidation/

Case Study: Blizzard Entertainment – Autoscaling for Overwatch Game Services

Context: Blizzard Entertainment (video game developer/publisher) has operated a private OpenStack cloud infrastructure since 2012 to host gaming services for millions of concurrent players. Overwatch is a best-selling team-based multiplayer shooter requiring consistent, responsive game server allocation to handle millions of simultaneous players with real-time performance requirements.

Problem: Running multiplayer video games in the cloud presented unique resource scheduling and management challenges:

- **Unpredictable demand spikes:** Game updates, new seasons, marketing campaigns, and esports events cause sudden traffic surges requiring rapid server provisioning
- **Real-time response requirements:** Unlike batch workloads, game servers cannot tolerate delays in resource allocation—players expect immediate game server provisioning on login
- **Waste prevention:** Over-provisioning servers during off-peak hours consumes budget and power; under-provisioning during peaks causes player disconnections and bad experience
- **Multi-region coordination:** Managing game servers across multiple data centers requires scheduling decisions that account for latency, availability zones, and player geographic distribution
- **Cost sensitivity:** Cloud gaming at massive scale means even small efficiency gains multiply across millions of players

The core challenge was implementing intelligent autoscaling in Nova/OpenStack to match resource allocation decisions to real-time game server demand patterns.

Solution: Blizzard implemented OpenStack autoscaling for Overwatch game services using:

1. OpenStack Heat Template-Driven Scheduling:

- Using Heat (OpenStack orchestration) to define application stacks with resource requirements
- Templates specify flavor (vCPU, RAM), image, and scheduling constraints
- Enables repeatable, version-controlled server provisioning through Heat API

2. Nova Flavor and Aggregates for Game Server Placement:

- Defining specific flavors optimized for game server workloads (e.g., dedicated cores, low-latency networking)
- Using Nova Host Aggregates to isolate game server hardware from other workloads
- Configuring AggregateInstanceExtraSpecsFilter to ensure game servers land on suitable hardware

3. Senlin Cluster Manager for Autoscaling:

- Using OpenStack Senlin (cluster life cycle management service) to define scaling policies
- Policies trigger based on Ceilometer metrics (CPU, memory, request queue depth)
- Automatic VM creation (via Nova scheduler) when demand increases
- Graceful VM termination when load decreases

4. Metrics-Driven Scaling Decision:

- Monitoring player queue depth, active game sessions, CPU utilization on existing servers
- Scaling policies defined as: "When queue_depth > 500 concurrent players, provision 50 new game servers"
- Nova scheduler allocates new VMs on available hosts using Placement service queries
- Terraform used for infrastructure-as-code management alongside Heat templates

5. Application-Level Graceful Shutdown:

- Game servers implement connection draining on shutdown (finish existing matches before terminating)
- Prevents player disconnections during scale-down events
- Nova scheduler can safely deallocate VMs after draining signals

Results/Lessons:

- **Real-time responsiveness:** Game servers provisioned within seconds of demand spike detection using Nova's fast VM scheduling
- **Cost optimization:** Autoscaling reduced idle server costs by 40-60% through predictive scaling based on historical patterns
- **Improved player experience:** No more "server full" rejections; autoscaling ensured capacity during launch events and peak times
- **Multi-region management:** Coordinated scheduling across regions using Senlin policies and Zaqr messaging

- **Operational simplicity:** Template-driven provisioning reduced manual server management overhead
- **Reliability:** Automatic failover and rescheduling when individual servers fail (Nova reschedules game servers to healthy hosts)

Key Insight: Blizzard demonstrates that **combining OpenStack's autoscaling components (Heat, Senlin, Nova, Placement) enables real-time resource management for latency-sensitive applications**. Unlike traditional batch systems that can tolerate provisioning delays, video game services demand that resource scheduling decisions be made in seconds. By using metrics-driven policies and Nova's fast VM scheduling with Placement service, Blizzard achieved responsive autoscaling that matches game server allocation to player demand in real-time. This shows that Nova's scheduler is effective not just for static workloads but for highly dynamic, responsive applications.

Reference: <https://www.openstack.org/videos/summits/denver-2019/how-blizzard-entertainment-uses-autoscaling-with-overwatch>

Proxy Servers: Reverse Proxy and Load Balancing

Case Study: Booking.com – Global Reverse Proxy Infrastructure for Scale

Context: Booking.com (1996–present) is one of the world's largest online travel platforms, processing millions of concurrent hotel reservations and user requests across 6 continents. With 220+ million searches processed daily, the company requires a high-performance reverse proxy infrastructure to handle incoming traffic from diverse global regions and distribute it across hundreds of backend servers.

Problem: Booking.com faced critical infrastructure challenges with global scale:

- **Request volume at scale:** Millions of simultaneous HTTP requests from global users needed to be efficiently routed to appropriate backend services
- **Single points of failure:** Without proper proxy infrastructure, backend servers were exposed to direct traffic spikes and would become bottlenecks
- **Latency across regions:** Users in different geographic regions experienced variable latency when connected to distant backend servers
- **SSL/TLS overhead:** Backend servers were consuming significant CPU resources encrypting and decrypting HTTPS traffic
- **DDoS vulnerability:** Direct exposure of backend origin servers made infrastructure vulnerable to targeted attacks
- **Traffic distribution complexity:** Different user request types (search, booking, payments) required routing to specialized backend pools

The company needed a reverse proxy solution that could handle massive concurrent connections while adding minimal latency.

Solution: Booking.com deployed **Nginx as a high-performance reverse proxy and load balancer** across multiple global edge locations:

1. Global Reverse Proxy Deployment:

- Deployed Nginx instances at internet exchange points and cloud provider data centers across Americas, Europe, Asia-Pacific, and Middle East
- Each geographic region maintains local Nginx proxy clusters to intercept user traffic before it reaches origin servers
- Users connect to the nearest Nginx reverse proxy, which then routes requests to appropriate backend pools

2. Intelligent Request Routing:

- Configured Nginx upstream modules to define backend server pools for different service types (search engine, booking service, payment processing, user account management)
- Used Nginx's URI-based routing rules to direct requests like `/search/*` to search pool, `/book/*` to booking pool
- Implemented request header inspection for routing authenticated vs. anonymous users to different backend tiers

3. Load Balancing with Health Checks:

- Nginx health checks continuously probe backend servers (TCP checks, HTTP GET requests to `/health` endpoint)
- Failed or slow servers are automatically marked unavailable; traffic reroutes to healthy servers
- Servers recovering from failures are gradually re-enabled using Nginx's gradual weight restoration
- Prevents cascading failures where one backend slowdown causes load to shift to already-stressed servers

4. SSL/TLS Termination:

- All HTTPS connections from users terminate at Nginx proxy layer
- Nginx handles certificate management, TLS handshakes, and encryption/decryption
- Backend servers communicate with Nginx via unencrypted HTTP, freeing backend CPU for application logic
- Reduced backend server CPU consumption by 30-40% by offloading cryptographic operations

5. Connection Pooling and Keepalive:

- Nginx maintains persistent HTTP connections (keep-alive) to backend servers
- Reuses established connections across multiple client requests, reducing handshake overhead
- Connection pooling reduces latency by 50-70% compared to establishing new backend connections per client request
- Implements upstream connection limits to prevent backend server socket exhaustion

6. Caching Layer at Proxy:

- Nginx caches frequently accessed responses (hotel listing pages, images, static JavaScript)
- Cache headers from backend services determine cache duration (30 seconds for listings, 1 hour for static assets)
- Local Nginx caches reduce backend server load by 40-60% for read-heavy workloads
- Significantly reduces response time for cached content (milliseconds vs. hundreds of milliseconds)

7. Rate Limiting and DDoS Protection:

- Nginx limit_req module enforces per-IP rate limits to prevent request floods
- Blocks traffic from IPs exceeding 100 requests/second
- Implements geolocation-based rules to drop traffic from known malicious IP ranges
- Nginx's memory-efficient request tracking enables handling attacks at edge before reaching backend

Results/Lessons:

- **99.99% availability:** With Nginx as reverse proxy absorbing traffic spikes and distributing load, backend infrastructure achieved 99.99% uptime SLA
- **Massive concurrency handling:** Nginx processes 2-3 million concurrent connections per server with low memory footprint (each connection ~1KB)
- **50% latency reduction:** By terminating SSL and connection pooling, average user request latency decreased from 600ms to 300ms globally
- **Backend protection:** Backend servers never exposed to public internet; DDoS attacks target Nginx proxies instead, which have additional hardening
- **Cost efficiency:** Reduced backend server count by 30% through better resource utilization and caching, significantly lowering infrastructure costs
- **Global scalability:** Added new geographic regions by simply deploying new Nginx clusters; no changes to origin server infrastructure needed
- **Industry validation:** Nginx became standard infrastructure pattern at Booking.com, adopted by team after team for new services

Key Insight: Booking.com demonstrates that **reverse proxy infrastructure using Nginx acts as a critical scaling and availability layer in globally distributed systems**. By intercepting all client traffic at Nginx proxies rather than sending it directly to origin servers, companies can achieve better reliability (health-checking and automatic failover), performance (connection pooling, caching, SSL offload), and security (DDoS mitigation, origin server protection). Nginx's efficient C-based architecture enables a single proxy server to handle millions of concurrent connections, making it ideal for massive internet-scale platforms handling global traffic.

Reference: <https://www.nginx.com/resources/case-studies/booking/>

Case Study: Dropbox – Reverse Proxy for API Rate Limiting and Request Routing

Context: Dropbox (2008–present) provides cloud file storage and synchronization services to 600+ million users across 180+ countries. The Dropbox API processes billions of file operations daily—uploads, downloads, sharing, and sync requests—each requiring careful request routing, rate limiting, and authentication.

Problem: Dropbox's API infrastructure faced challenges managing global scale and usage patterns:

- **Per-user rate limiting:** Different user tiers (free, professional, business) have different rate limits; enforcing these limits on backend servers was inefficient
- **Request authentication complexity:** Every API request required token validation against authentication services; validating at each backend added latency
- **API version routing:** Dropbox maintains multiple API versions; routing requests to appropriate version-specific backends was complex
- **User quota enforcement:** Enforcing individual storage quotas and bandwidth limits needed to happen early in request pipeline to fail fast
- **Backend server protection:** Backend services shouldn't need to know about rate limit policies; proxies should enforce limits transparently
- **Request deduplication:** Retried requests from clients needed deduplication to prevent duplicate operations on backend

The challenge was implementing transparent request routing and enforcement policies without modifying backend service code.

Solution: Dropbox deployed **Nginx as an API gateway and reverse proxy** to handle request routing, rate limiting, and policy enforcement:

1. Request Authentication at Proxy Layer:

- Nginx intercepts all API requests and extracts authentication tokens from request headers
- Uses Nginx subrequests to validate tokens with centralized auth service (typically responds in <10ms)
- Only authenticated requests are forwarded to backend; invalid tokens rejected at proxy with 401 response
- Prevents backend services from processing invalid requests, saving backend resources

2. Per-User Rate Limiting:

- Nginx's `limit_req_status` and `geo` modules implement fine-grained rate limiting
- Rate limits keyed by user ID (extracted from auth token), not IP address
- Different limits for different user tiers: free users (10 req/sec), professional (100 req/sec), business (1000 req/sec)

- Throttled requests queued in Nginx (up to 100 queued) before being rejected, allowing request bursts to be smoothed

3. API Version Routing:

- Nginx analyzes request path to determine API version (/1.0/ vs /2.0/ endpoints)
- Routes requests to appropriate backend pool: v1 endpoints to legacy servers, v2 to modern services
- Enables running multiple API versions simultaneously with independent backend infrastructure
- Allows gradual API migration without forcing all users to upgrade simultaneously

4. Quota Enforcement and Fail-Fast:

- Nginx queries a Redis cache of user quotas (updated every minute from source of truth)
- Checks if user has remaining storage quota before forwarding request to backend
- Rejects requests from quota-exceeded users with 507 (Insufficient Storage) response immediately
- Prevents backend from processing requests that would fail anyway, reducing wasted work

5. Request Deduplication:

- Nginx maintains a distributed request cache keyed by (user_id, request_body_hash, idempotency_key)
- For file operations with idempotency keys, returns cached response if same request detected within 60 seconds
- Handles client retries transparently—backend only sees first request, user gets same response without duplication
- Prevents double-uploads or double-deletes caused by network retries

6. Error Rate and Circuit Breaking:

- Nginx tracks error rates from each backend server in real-time
- If a backend service returns errors on >10% of requests for 30 seconds, Nginx circuit breaker trips
- Failed backend is temporarily removed from load balancing rotation; requests sent to healthy backends
- Failed backend is gradually re-enabled after error rate drops below 5%, preventing thundering herd

7. Request Logging and Metrics:

- Nginx logs every request (user ID, timestamp, request path, response status, latency) to local files
- Logs are streamed to central logging infrastructure for billing, analytics, and debugging
- Real-time metrics (requests/sec, error rate, 99th percentile latency) exported to monitoring systems
- Enables tracking per-user usage patterns and detecting abuse

Results/Lessons:

- **API reliability:** Rate limiting and circuit breaking at Nginx layer increased backend stability; incident rate dropped 40%
- **Transparent enforcement:** Policy changes (e.g., increasing rate limit for a user tier) require only Nginx config change, no backend modifications
- **Request deduplication eliminated:** Double-writes and double-deletes from retried requests became effectively zero
- **Authentication latency eliminated:** By validating tokens at proxy, backend services no longer needed to perform auth, reducing per-request latency
- **Cost efficiency:** Fail-fast quota checking and deduplication prevented wasted backend processing; reduced backend server count by 25%
- **Operational flexibility:** Can route different user cohorts to different backend versions for A/B testing without client awareness
- **Scale capability:** Single Nginx proxy server handles 100,000+ concurrent API connections with sub-millisecond request routing latency

Key Insight: Dropbox demonstrates that **reverse proxies like Nginx are essential for modern API platforms**, enabling transparent enforcement of policies (rate limiting, quotas, authentication) that would be expensive or complex to implement in backend services. By implementing policy enforcement at the proxy layer rather than in application code, companies achieve better performance (fail-fast rejection), consistency (policies enforced uniformly), and operational flexibility (policy changes without service redeployment). Nginx's event-driven architecture makes it ideal for API gateways handling millions of concurrent requests with minimal overhead.

Reference: <https://dropbox.tech/infrastructure/building-faster-dropbox-sync-engine>